

Select Transformer Models

John Snyder

STAT 9340: Spring 2023

Overview

Transformers have revolutionized many modern applications

- BERT (Bidirectional Encoder Representations from Transformers): Developed by Google for NLP tasks
 - ▶ Useful for all kinds of tasks
- GPT (Generative Pre-trained Transformer): Developed by OpenAI for generating natural language text.
 - ▶ Input text, the model will continue predicting tokens
- T5 (Text-to-Text Transfer Transformer): Developed by Google that can perform a wide range of NLP tasks.
 - ▶ Text classification, translation, Q&A
- DALL-E: Generates images from text
- ViT: Vision transformers use attention to learn local features in images. Surpassed most cutting edge CNN models.
- TFT (temporal fusion transformers): Uses attention for time series forecasting.

Section 1

BERT

Introduction to BERT

Bidirectional **E**ncoder **R**epresentations from **T**ransformers (BERT) is ~2018 natural language processing model developed by Google AI researchers. It was the first of a wave of new transformer based models that revolutionized the NLP field by offering significant improvements in various tasks such as sentiment analysis, question-answering, and named entity recognition.

Keys:

- Bidirectional Context: BERT uses a bidirectional approach to understand context using the entire sentence.
- Pre-training and Fine-tuning: BERT is pre-trained by Google and the weights are shared publically. This model can then be fine-tuned for specific tasks with relatively small datasets.
 - ▶ This is **HUGE**

Training of BERT

These models are massive and pre trained versions of various sizes available from Google. Both were trained on large corpus' of words in the English Wikipedia and a collection of 10s of thousands of books.

BERT Base:

- 12 transformer blocks
- 110 million parameters
- Trained over 4 days on 4 cloud TPUs (tensor processing units)

BERT Large

- 24 transformer blocks
- 336 million parameters
- Trained over 4 days on 16 cloud TPUs

BERT only uses the encoder part of the transformer architecture discussed previously and does not have a decoder, making it able to capture bidirectional contextual information in the input sequence. This means it is designed to perform tasks where the output is not an arbitrary length

BERT Training Details

BERT is trained using two tasks

① Masked word prediction

Here, some of the input tokens are randomly ($\sim 15\%$) masked, and the model is trained to predict the masked tokens based on the context of the surrounding tokens. This is where the *Bidirectional* portion of the model comes from, since context can be learned based on words on either side.

② Next sentence prediction

Here we are predicting whether two sentences are consecutive or not. Sentences are paired together such that there is a 50/50 chance of them being adjacent to one another.

During pre-training (i.e. Google's training process) both of these tasks are conducted simultaneously.

Masking Example

Suppose we have the following masked sentence

“I like to eat [MASK] with my cereal in the morning.”

In this sentence, the word “fruit” has been replaced with a special masking token “[MASK].” The model executes simply by adding a classification layer to the top of the standard encoder architecture to predict the missing word.

We may have the following probability distribution for predicting the missing word:

- pizza: 0.02
- sushi: 0.01
- banana: 0.43
- orange: 0.03
- kiwi: 0.01
- fruit: 0.50

It's slightly more complex in reality as BERT uses WordPiece tokenization (we will see this in the demo)

Next sentence prediction example

Here, the output of the model is a binary classification task predicting if two input sentences are consecutive or not. This allows the model to understand the logical connections between sentences.

These two sentences are not related

- Sentence A: "I'm planning to go on a trip to Europe."
- Sentence B: "The cat is sleeping in the sun."
- Label: NotConnected

And these two sentences are related

- Sentence A: "I love to eat pizza."
- Sentence B: "Pizza is my favorite food of all time."
- Label: Connected

In this case we have a binary output (1/0) indicating if sentence B is after sentence A.

Combining NSP and MLM

We have two tasks: NSP and MLM

The MLM loss function can be written as:

$$L_{MLM} = -E[(\log P(w_i | w_1, \dots, w_{i-1}, w_{i+1}, \dots, w_n))]$$

E is the expectation over the masked words, w_i is the masked word, and $P(w_i | w_1, \dots, w_{i-1}, w_{i+1}, \dots, w_n)$ is the conditional probability of predicting the masked word w_i given its context.

The NSP loss function can be written as:

$$L_{NSP} = -E[(\log P(y | S_1, S_2))]$$

During training, BERT combines the losses from both tasks to form its objective. The final loss function is a weighted sum of the MLM and NSP losses:

$$L_{total} = L_{MLM} + L_{NSP}$$

Crafting Inputs for BERT training

The pre-training of the BERT model effectively combines both MLM and NSP tasks into one input in the following way.

Every input to the model will be two sentences concatenated with a special [SEP] token. We then prepend another special [CLS] token (short for classification token). The resulting input looks like this:

[CLS] Sentence A [SEP] Sentence B [SEP]

From there, tokens from sentence A and B are randomly masked and the model is trained.

Note that the softmax function for classifying the adjacency of sentences is actually

$$P(y|S_1, S_2) = \text{softmax}(W * h[\text{CLS}] + b)$$

That is, $h[\text{CLS}]$ is the final hidden state of the [CLS] token which is used *only* for this purpose.

Fine Tuning

We can directly obtain a pre-trained BERT model which can be “fine-tuned” on specific tasks with smaller datasets. This fine-tuning process adapts BERT’s pre-trained knowledge(i.e. weights) to a user specified problem.

The pretrained model we obtain contains all weights up to the final hidden state of each token. From there, tasks can take many forms, but the common theme is that we will add some layers on top of that and then train a model with a relatively small set of data.

Occasionally, we may want to do the NSP and MLM tasks ourselves using a smaller set of domain specific text data. This can be viewed as a second pre-processing step before building a task specific model.

Some tasks:

- Text classification (sentiment, topics, etc)
- Named Entity Recognition: Find important people, places, etc in text
- Paraphrase detection: Check if two sentences have similar meaning.
- and more!

Section 2

GPT models

Introduction

Generative Pretrained Transformer (GPT) models are another class of LLM.

- BERT was based only on the *encoder* portion of the transformer model.
- GPT models are based solely on the *decoder* portion.
 - ▶ This is because the task for GPT models is token by token generation

GPT models have made waves in popular culture in recent months and they are both flexible and powerful. They owe this in part to the ability of the transformer architecture to understand context very well, and also the fact that the models/training data are enormous.

For example GPT3 has

- 175 **billion** parameters (BERT large was 336 million)
- Was trained on most of the public internet

When we understand the scale these models exist on, some of the magic of their perceived operation gets removed.

Rumors are that GPT4 has ~100 **trillion** parameters.